

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 6

Roll no: 54 PRN No: 12414148 Name of Student: Sanika Kulkarni

Subject Name: Deep Learning

Problem Statement:

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm:

1. Start the program.
2. Import the required libraries such as TensorFlow, Keras, NumPy, Matplotlib, Scikit-learn, and OS.
3. Download and load the potato disease image dataset from Kaggle.
4. Set the image size, batch size, and random seed for dataset processing.
5. Split the dataset into training and validation sets using an 80:20 ratio.
6. Display sample images and their class labels from the training dataset.
7. Preprocess and optimize the datasets using rescaling, caching, shuffling, and prefetching.
8. Calculate the number of images in each class and determine class weights to handle class imbalance.
9. Build the CNN model using convolution, pooling, fully connected, dropout, and output layers.
10. Compile the model using the Adam optimizer, sparse categorical cross-entropy loss, and accuracy metric.
11. Train the CNN model on the training dataset for the specified number of epochs using class weights.
12. Evaluate the model on the validation dataset and plot training and validation accuracy and loss.
13. Generate predictions for the validation images using the trained model.

14. Calculate accuracy, precision, recall, F1-score, and display the confusion matrix to analyze classification performance.
15. Display the final results and stop the program.

Code:

Using Potato Disease dataset: The potato disease dataset contains 2152 images and is separated into three different labels such as 'early bright', 'late bright', and 'healthy' leaf.

```
import kagglehub
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
from sklearn.utils.class_weight import compute_class_weight
import os
path = kagglehub.dataset_download(
    "faysalmiah1721758/potato-dataset"
)
print ("Path to dataset files:", path)
BATCH_SIZE = 32
IMG_SIZE = (128, 128)
SEED = 123
train_ds = tf.keras.utils.image_dataset_from_directory(
    path,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

val_ds = tf.keras.utils.image_dataset_from_directory(
    path,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
class_names = train_ds.class_names
print("Detected classes:", class_names)
plt.figure(figsize=(10, 10))
for images, labels in train_ds.take(1):
    for i in range(9):
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
```

```

        plt.title(class_names[labels[i]])
        plt.axis("off")
plt.tight_layout()
plt.show()
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().shuffle(1000).prefetch(
    buffer_size=AUTOTUNE
)
val_ds = val_ds.cache().prefetch(
    buffer_size=AUTOTUNE
)
class_counts = []
for class_name in class_names:
    class_path = os.path.join(path, class_name)
    count = len([
        file for file in os.listdir(class_path)
        if os.path.isfile(os.path.join(class_path, file))
    ])
    class_counts.append(count)
print("Class counts:", class_counts)
num_classes = len(class_names)
model = models.Sequential([
    layers.Input(shape=(128, 128, 3)),
    layers.Rescaling(1./255),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()
EPOCHS = 10
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS,
    class_weight=class_weights
)
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
plt.figure(figsize=(7, 5))

```

```

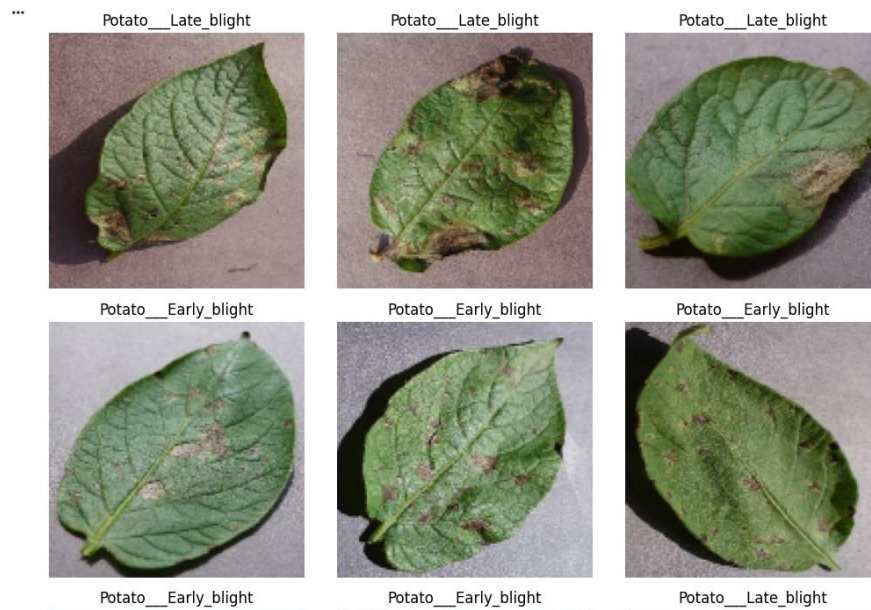
plt.plot(acc, label='Train Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.show()
loss = history.history['loss']
val_loss = history.history['val_loss']
plt.figure(figsize=(7, 5))
plt.plot(loss, label='Train Loss')
plt.plot(val_loss, label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.show()
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
import numpy as np
import matplotlib.pyplot as plt
y_true = []
y_pred = []
for images, labels in val_ds:
    predictions = model.predict(images, verbose=0)
    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(predictions, axis=1))
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='weighted')
recall = recall_score(y_true, y_pred, average='weighted')
f1 = f1_score(y_true, y_pred, average='weighted')
print(f"Accuracy : {accuracy * 100:.2f}%")
print(f"Precision: {precision * 100:.2f}%")
print(f"Recall   : {recall * 100:.2f}%")
print(f"F1 Score : {f1 * 100:.2f}%")
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6, 5))
plt.imshow(cm, cmap='Blues')
plt.xticks(range(len(class_names)), class_names, rotation=45)
plt.yticks(range(len(class_names)), class_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
for i in range(len(class_names)):
    for j in range(len(class_names)):
        plt.text(j, i, cm[i, j], ha='center', va='center')
plt.tight_layout()
plt.show()

```

Output:

Dataset:



Model Summary:

Model: "sequential"

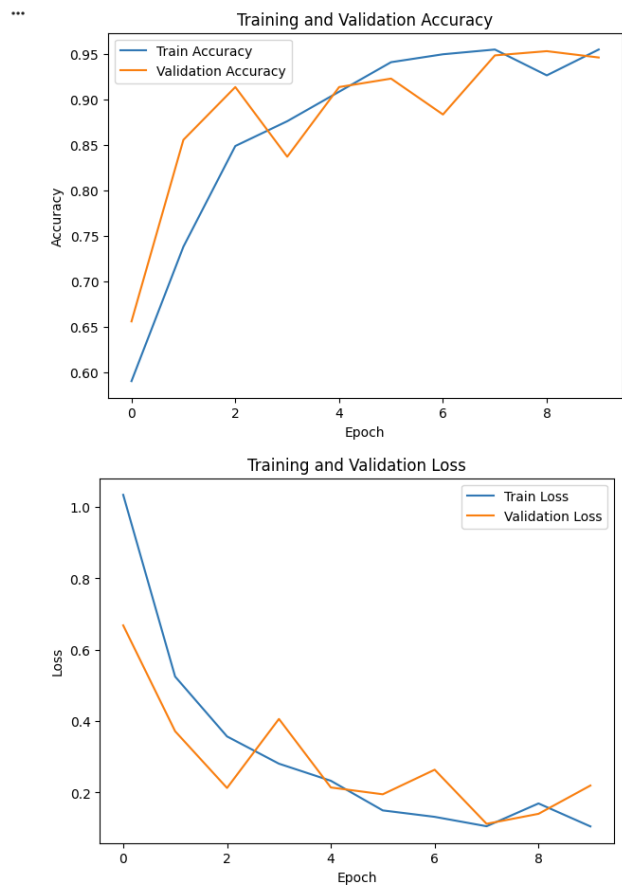
Layer (type)	Output Shape	Param #
rescaling (Rescaling)	(None, 128, 128, 3)	0
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

Total params: 3,305,027 (12.61 MB)

Trainable params: 3,305,027 (12.61 MB)

Non-trainable params: 0 (0.00 B)

Validation Accuracy and Loss Graphs:



Evaluation Metrics:

... Accuracy : 94.65%
Precision: 94.97%
Recall : 94.65%
F1 Score : 94.66%

